

Better Probe Sequences

Quadratic Probing and Double Hashing

Unit III - Lecture 29

Quadratic probing step by i^2 instead of i

Double hashing a second hash function picks the step

Recap: linear probing's weakness, and the fix to try

Lecture 28: linear probing steps by 1 each time (idx , $\text{idx}+1$, $\text{idx}+2$, ...). This caused SECONDARY CLUSTERING - key 24 needed 3 hops because 9's earlier collision left a solid block of occupied slots in its path.

Today's idea: what if the STEP SIZE itself changed, so probe sequences do not pile up in a straight line?

- Quadratic probing: step by 1, then 4, then 9, then 16 (i -squared) - jumps grow quickly, breaking up straight-line pileups.
- Double hashing: the STEP SIZE itself comes from a second hash function - two different keys with the same home index usually take completely different paths.

Quadratic probing: jump by i-squared

`idx = (home + i*i) % tableSize, for i = 0, 1, 2, 3, ...`

```
int idx, i = 0;
while (i < TABLE_SIZE)
{
    idx = (home + i * i) % TABLE_SIZE;
    if (table[idx] == EMPTY)
    {
        table[idx] = key;
        return idx;
    }
    i++;
}
```

- $i=0$: check home itself.
- $i=1$: home+1. $i=2$: home+4. $i=3$: home+9. The gaps GROW, instead of staying fixed at 1 like linear probing.
- Growing gaps mean two colliding keys are unlikely to keep landing on each other's later probes.

Quadratic probing

0	1	2	3	4	5	6
.	15	23	17	24	12	9

key	home	probe sequence tried	landed at
9	2	$i=0:\text{idx2}(\text{full}), i=1:\text{idx3}(\text{full}), i=2:\text{idx6}(\text{empty})$	index 6
24	3	$i=0:\text{idx3}(\text{full}), i=1:\text{idx4}(\text{empty})$	index 4

Compare with linear probing: 24 needed 3 hops there (landed at index 6). Here it needs only 2 checks and lands at index 4 - quadratic probing genuinely reduced its cost, on this exact key.

The catch: quadratic probing can fail before the table is full

With `tableSize = 8` (NOT prime), the values of $i*i \% 8$ only ever produce $\{0, 1, 4\}$ - just 3 distinct offsets, no matter how large i grows.

0	1	2	3	4	5	6	7
8	16	.	.	24	.	.	.

insert(32) FAILS here - even though 5 of 8 slots are still empty.

- Keys 8, 16, 24, 32 all share home index 0 (each is a multiple of 8).
- 8, 16, 24 land at indices 0, 1, 4 - exactly the ONLY offsets $\{0,1,4\}$ reachable from home 0.
- 32's probe sequence just keeps revisiting 0, 1, 4 forever - it can never reach the 5 genuinely empty slots.

Double hashing: a second hash function picks the step

```
idx = (home + i * h2(key)) % tableSize, h2(key) = 1 + (key % (tableSize-1))
```

```
int h1 = key % TABLE_SIZE;
int step = 1 + (key % (TABLE_SIZE-1));
int idx, i = 0;
while (i < TABLE_SIZE)
{
    idx = (h1 + i * step) % TABLE_SIZE;
    if (table[idx] == EMPTY)
    {
        table[idx] = key;
        return idx;
    }
    i++;
}
```

- step depends on the KEY, not just on i - two keys with the same home almost always get different step sizes.
- The "+1" in h2 guarantees step is never 0, so the probe sequence always moves.
- Choosing tableSize prime again guarantees every slot is eventually reachable.

Double hashing: verified

0	1	2	3	4	5	6
.	15	23	17	24	12	9

key	h1 (home)	h2 (step)	landed at index
9	2	4	6
24	3	1	4

search(9) probe count: linear = 3, quadratic = 3, double hashing = 2.
search(24) probe count: linear = 4, quadratic = 2, double hashing = 2.
Double hashing matches or beats both earlier methods on every key here.

All three methods, side by side

Aspect	Linear	Quadratic	Double hashing
Step pattern	always +1	+1, +4, +9, ...	+step(key) each time
Clustering risk	primary AND secondary	secondary only, reduced	lowest of the three
Can it fail early?	no (with room left)	yes, if tableSize not prime	no (if tableSize prime)
search(9) probes here	3	3	2
search(24) probes here	4	2	2

Change the step, not just the slot

- Quadratic probing: $\text{idx} = (\text{home} + i*i) \% \text{tableSize}$ - growing gaps reduce secondary clustering, verified: 24 improved from 3 hops to 2.
- Quadratic probing's catch: with a non-prime tableSize, only some offsets are ever reachable - insert can FAIL even with empty slots remaining. Verified directly: insert(32) failed with 5 of 8 slots still free.
- Double hashing: $\text{idx} = (\text{home} + i*\text{step}(\text{key})) \% \text{tableSize}$ - the step itself depends on the key, giving the lowest clustering of the three methods tested.
- All three need tableSize chosen carefully - prime sizes avoid the failure modes of both quadratic probing and double hashing.

EXIT TICKET tableSize = 7. Insert 5, 12, 19 using quadratic probing (home = key % 7). Show each probe sequence and the final table.